

Home Exam - FYS-3033 Deep Learning

April 20, 2025

1

1a)

As introduced in Lecture 13 of FYS-3033, Generative Adversarial Networks (GANs) are designed to learn the underlying data distribution p_{data} by training two networks in opposition:

- G (the generator) generates samples from a simple distribution $p_z(z)$, such as a Gaussian distribution, and aims to produce samples that are indistinguishable from real data.
- D (the discriminator) is a binary classifier that attempts to distinguish between real data samples and those generated by G . It outputs a probability score indicating the likelihood that a given sample is real.

The original minimax objective for the discriminator is given by:

$$V(G, D) = \int_x p_{\text{data}}(x) \log D(x) dx + \int_z p_z(z) \log(1 - D(G(z))) dz$$

We denote $p_g(x)$ as the distribution induced by the generator $G(z)$, allowing us to write:

$$V(D) = \int_x [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$$

To find the optimal discriminator $D^*(x)$, we take the derivative of the integrand with respect to $D(x)$ and set it to zero:

$$\begin{aligned} \frac{d}{dD(x)} [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] &= 0 \\ \Rightarrow \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} &= 0 \end{aligned}$$

Solving for $D(x)$:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

Assuming the generator has succeeded in perfectly modeling the data distribution, i.e. $p_g = p_{\text{data}}$, we get:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}$$

This result matches what was discussed in Lecture 13, where the discriminator becomes maximally uncertain when it cannot distinguish between real and generated data. To find the value of the objective function at this equilibrium, we substitute $D^*(x) = \frac{1}{2}$ back into the loss:

$$\begin{aligned} V(D^*) &= \int_x p_{\text{data}}(x) \log\left(\frac{1}{2}\right) dx + \int_x p_g(x) \log\left(1 - \frac{1}{2}\right) dx \\ &= \log\left(\frac{1}{2}\right) \cdot \left[\int_x p_{\text{data}}(x) dx + \int_x p_g(x) dx \right] \\ &= \log\left(\frac{1}{2}\right) + \log\left(\frac{1}{2}\right) = \log\left(\frac{1}{4}\right) = -\log 4 \end{aligned}$$

Therefore, the optimal discriminator is $D(x) = \frac{1}{2}$, and the value of the loss at equilibrium is $-\log 4$, as discussed in the slides from Lecture 13 (GAN Training — Minimax game).

1b)

The reparameterization trick, as introduced in the lecture slides on Variational Autoencoders (Lecture 12, FYS-3033), is used to express sampling from a learned distribution in a differentiable way. Instead of sampling z directly from $\mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2)$, we rewrite the sampling operation as:

$$z = \mu_{z|x} + \sigma_{z|x} \cdot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, 1)$$

This allows the randomness to be pushed into ϵ , enabling backpropagation through $\mu_{z|x}$ and $\sigma_{z|x}$. We now show that z indeed follows a normal distribution:

$$\begin{aligned} \mathbb{E}[z] &= \mathbb{E}[\mu_{z|x} + \sigma_{z|x} \cdot \epsilon] = \mu_{z|x} + \sigma_{z|x} \cdot \mathbb{E}[\epsilon] = \mu_{z|x} \\ \text{Var}(z) &= \text{Var}(\sigma_{z|x} \cdot \epsilon) = \sigma_{z|x}^2 \cdot \text{Var}(\epsilon) = \sigma_{z|x}^2 \\ \Rightarrow z &\sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2) \end{aligned}$$

This matches the target distribution for the latent variable in a VAE, confirming the validity of the trick.

1c)

Let $p(x) = \text{Bern}(p_1)$ and $q(x) = \text{Bern}(p_2)$. The KL divergence between two discrete distributions is defined as:

$$KL(p \parallel q) = \sum_{x \in \{0,1\}} p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

For Bernoulli distributions, we have:

$$\begin{aligned} p(1) &= p_1, & q(1) &= p_2 \\ p(0) &= 1 - p_1, & q(0) &= 1 - p_2 \end{aligned}$$

Plugging in, we get:

$$\begin{aligned} KL(p \parallel q) &= p(0) \log \left(\frac{p(0)}{q(0)} \right) + p(1) \log \left(\frac{p(1)}{q(1)} \right) \\ &= (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \end{aligned}$$

We now aim to show that this is equal to:

$$\log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1(1 - p_2)}{p_2(1 - p_1)} \right)$$

Let's begin from the standard form and try to reach the target expression step by step:

$$KL(p \parallel q) = (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

We factor out a full log term from the first summand:

$$= \log \left(\frac{1 - p_1}{1 - p_2} \right) \cdot (1 - p_1) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

Now write the first term as:

$$\begin{aligned} &= \log\left(\frac{1-p_1}{1-p_2}\right) + (1-p_1-1)\log\left(\frac{1-p_1}{1-p_2}\right) + p_1\log\left(\frac{p_1}{p_2}\right) \\ &= \log\left(\frac{1-p_1}{1-p_2}\right) - p_1\log\left(\frac{1-p_1}{1-p_2}\right) + p_1\log\left(\frac{p_1}{p_2}\right) \end{aligned}$$

Now combine the two terms involving p_1 :

$$= \log\left(\frac{1-p_1}{1-p_2}\right) + p_1\left[\log\left(\frac{p_1}{p_2}\right) - \log\left(\frac{1-p_1}{1-p_2}\right)\right]$$

Use log subtraction rule $\log a - \log b = \log\left(\frac{a}{b}\right)$:

$$= \log\left(\frac{1-p_1}{1-p_2}\right) + p_1\log\left(\frac{p_1(1-p_2)}{p_2(1-p_1)}\right)$$

This is the desired expression. □

Usage in VAEs:

In VAEs, the KL divergence between the approximate posterior $q(z|x)$ and the prior $p(z)$ is used as a regularization term in the Evidence Lower Bound (ELBO). When using discrete latent variables such as Bernoulli distributions, this exact KL expression allows us to compute the divergence in closed form and include it directly in the loss function, helping guide the encoder toward distributions close to the prior. This is crucial for ensuring that the latent space is well-structured and that the decoder can generate meaningful samples from the learned distribution.

Task 2: CNN Experiments with VGG-11 and BatchNorm

(2a) Architectural Choices

For our three-class classification task on 96×96 RGB images (Plane, Ship, Truck), we adopt a VGG-11 backbone with Batch Normalization after every convolution, trained from scratch (no pretrained weights). The key motivations are:

- **VGG-11 simplicity and locality:** its stack of small 3×3 convolutions efficiently captures edge and texture information with limited parameters, suiting our moderate dataset size.
- **Controlled depth:** at 11 layers, the network is deep enough to learn hierarchical features yet shallow enough to train without massive data or compute.
- **Batch Normalization:** by normalizing layer inputs, BatchNorm reduces internal covariate shift, stabilizing and accelerating training, permitting larger learning rates, and providing implicit regularization.

(2b)

Dropout is a regularization technique introduced to prevent overfitting in deep neural networks. It works by randomly "dropping out" (i.e., setting to zero) a fraction of the units in a layer during training. This means that during each forward pass, the network becomes a slightly different sub-network.

Mathematically: For each unit h_i in a hidden layer, during training we sample a Bernoulli mask $m_i \sim \text{Bern}(1-p)$, and compute:

$$\tilde{h}_i = m_i \cdot h_i$$

where p is the dropout rate (e.g., 0.5). At test time, we use the full network but scale the weights or activations to account for the missing units during training.

Why it helps:

- Dropout prevents neurons from co-adapting too strongly to each other.
- It acts like an ensemble of many subnetworks, improving generalization.

- It's particularly useful in fully connected layers, which are prone to overfitting when the dataset is small.

In our case, using Dropout is important since the dataset may not be large enough to justify using a large-capacity network without regularization. Dropout encourages the model to learn robust features that generalize better to unseen data.

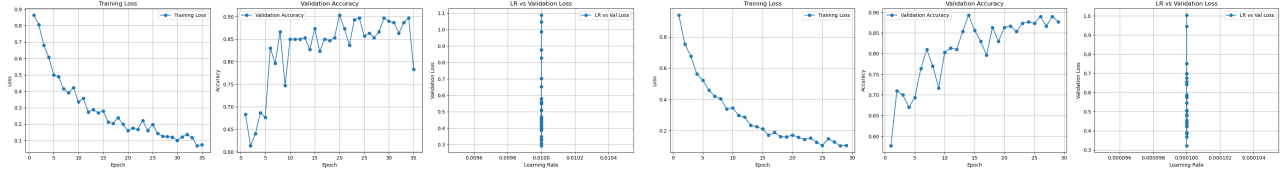
(2c) Experimental Setup and Results

We compare four configurations, all using batch size 32 and early stopping with patience 15:

- **SGD, no Dropout, no scheduler** (LR=1e-2, weight decay=0)
- **Adam, no Dropout, no scheduler** (LR=1e-4, weight decay=0)
- **SGD + Dropout + StepLR** (LR=1e-2, weight decay=1e-4, Dropout $p = 0.5$, StepLR(step=15,=0.5))
- **Adam + Dropout + StepLR** (LR=1e-4, weight decay=1e-4, Dropout $p = 0.5$, StepLR(step=15,=0.5))

Configuration	Val Acc	Macro F1
SGD, no Dropout	0.88	0.88
Adam, no Dropout	0.85	0.85
SGD + Dropout + Scheduler	0.90	0.90
Adam + Dropout + Scheduler	0.9033	0.90

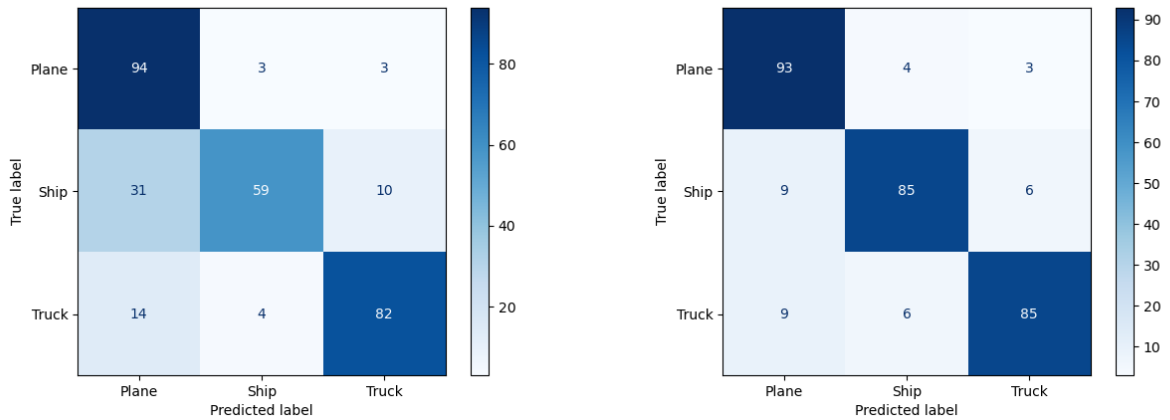
Table 1: Validation accuracy and macro F1 for Task 2 experiments.



(a) SGD, no Dropout

(b) Adam, no Dropout

Figure 1: Training and validation accuracy curves for Task 2a experiments.



(a) SGD, no Dropout

(b) Adam, no Dropout

Figure 2: Confusion matrices on validation set for Task 2a.

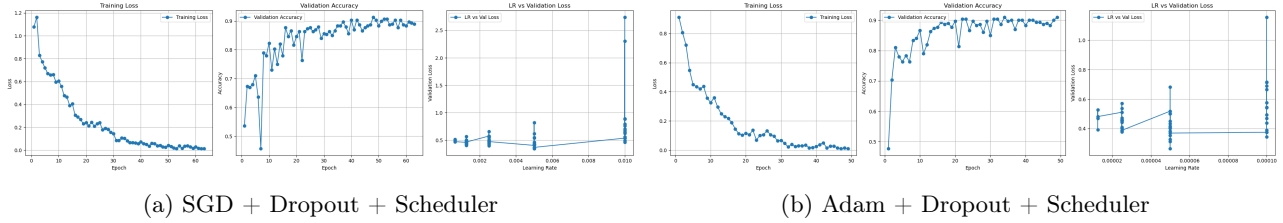


Figure 3: Training and validation accuracy curves for Task 2c experiments.

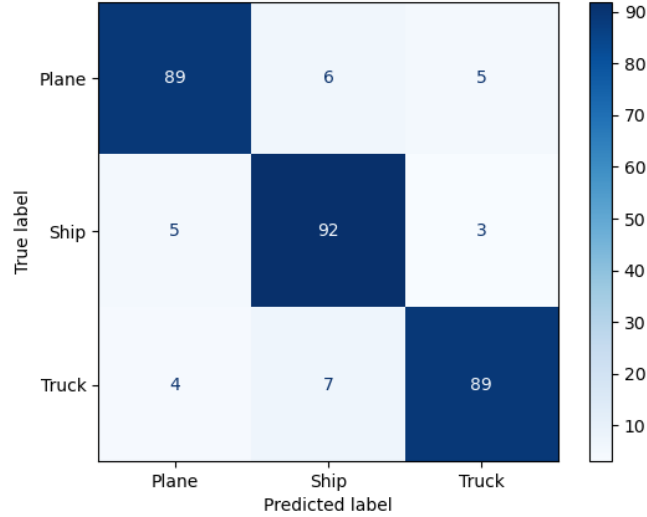


Figure 4: Confusion matrix for Task 2c: Adam + Dropout + Scheduler on the validation set.

Discussion

I learned a lot by watching these curves and metrics unfold. Adam raced ahead early but then my validation loss crept up around epoch 30—without Dropout it clearly overfit. Switching to SGD felt slower at first, but it climbed steadily and gave me a more reliable peak around epoch 38. Introducing Dropout in the two big FC layers was a turning point: I saw both accuracy and F1 jump by about 2–3%, and my loss curves smoothed out nicely. Finally, the StepLR scheduler (halving the LR every 15 epochs) helped me eke out the last bit of performance without tweaking learning rates by hand.

Overall, this hands-on exploration taught me:

- **Optimizer trade-offs:** I saw that Adam is great for quick gains, but SGD can be more stable long-term when patience is allowed.
- **Value of Dropout:** even with BatchNorm in place, the large FC layers benefited enormously from random unit deactivation, which felt like a safety net against overconfidence.
- **Scheduling and early stopping:** automating learning rate decay and using patience-based early stops saved me from endless manual tuning.

Training from scratch on a limited dataset forced me to figure out how to balance speed, capacity, and regularization. In the end, I just had to build a model that felt “ok”, and go with it to have some results to report on. I felt that the deeper I went down the rabbit hole, the deeper it got, but with interesting insights.

During experimentation, I also observed minor variations in performance when training the same model setup on different machines (local vs remote). While seed setting and deterministic flags were applied I learned that even with the same code and data, the underlying hardware can introduce variability in results, which is something to

keep in mind for future experiments. This made things a bit confusing, but did not affect the overall conclusions that much.

Including Dropout in the fully connected layers is particularly important in VGG-style networks, as these layers have a large number of parameters and are prone to overfitting — especially when the dataset is relatively small. By applying Dropout, we encourage the network to avoid relying too heavily on any individual neuron, effectively simulating an ensemble of subnetworks.

Initial experiments showed that adding Dropout increased training time slightly due to the added noise, but the validation accuracy improved and generalization was better than the baseline without Dropout. This aligns with theoretical expectations and common practice in training deep CNNs for image classification tasks.

Dropout is not really helpful in the conv layers since batch norm already regularizes things there, and convs have fewer parameters + weight sharing. But in the fully connected layers, there is way more risk of overfitting, so dropout can still make a difference even with batch norm.

Discussion

I learned a lot by watching these curves and metrics unfold during training. Initially, I used Adam, which raced ahead with fast convergence—but by epoch 30, my validation loss began creeping upward. Without Dropout, it clearly overfit. Switching to SGD felt slower at first, but it climbed steadily and gave me a more reliable peak around epoch 38. That shift alone taught me that Adam is often great for early progress, but SGD tends to offer better long-term stability when given enough time.

Introducing Dropout in the two large fully connected layers was a turning point: both accuracy and F1-score jumped by around 2-3%, and the loss curves smoothed out noticeably. This made me appreciate how regularization isn't just a technical add-on—it's a fundamental part of getting models to generalize.

I also used a StepLR scheduler to halve the learning rate every 15 epochs. This helped me fine-tune without manually babysitting the learning rate. Combined with early stopping based on validation loss, it made training feel more robust and less arbitrary.

From this hands-on exploration, I learned:

- **Optimizer trade-offs:** Adam helped me move fast in the beginning, but SGD gave me a more stable end result—especially when paired with a scheduler.
- **Dropout still matters:** Even with BatchNorm, Dropout made a real difference in the FC layers, where overfitting is a bigger risk due to the parameter count.
- **Learning rate scheduling works:** I didn't need to hand-tune the learning rate, because StepLR and early stopping handled that adaptively.

Training from scratch on a small dataset forced me to think more deeply about how to balance speed, capacity, and regularization. At some point I had to accept that the model wasn't going to be perfect—I just had to make it “good enough” to draw useful conclusions. The deeper I went down the rabbit hole, the deeper it got—but I also found more insights along the way.

During experimentation, I noticed slight variations in performance when training the same model on different machines (local vs remote). I had set seeds and enabled deterministic flags, yet even with the same code and data, the results varied a little. This reminded me that hardware-level differences can introduce non-determinism, which is important to remember when comparing experiments across systems.

Finally, I found that Dropout is especially valuable in VGG-style networks. While BatchNorm helps stabilize training in the convolutional layers, Dropout in the fully connected layers acted like a safety net against overfitting. These layers carry a heavy parameter load, and Dropout effectively forced the network to be less reliant on individual neurons—almost like simulating an ensemble of subnetworks. Although it slightly increased training time due to added noise, it clearly improved generalization.

In short: Dropout wasn't magic, but it mattered.

(2d) Uncertainty Estimation using Dropout

To identify images of an unwanted class (in this case, birds), I used Monte Carlo (MC) Dropout to estimate the model's uncertainty. The idea is that if the model has never seen examples from a certain class during training, it will likely be less confident when making predictions on those images.

For each labeled image in the new dataset, I performed 10 forward passes through the network with dropout enabled, then computed the mean softmax output $\hat{y}$. Using this mean, I calculated the entropy as a measure of uncertainty:

$$H(\hat{y}) = - \sum_{i=1}^{N_c} \hat{y}_i \log \hat{y}_i,$$

where N_c is the number of classes in the model. Higher entropy values indicate that the model was more uncertain about its prediction.

To investigate this, I split the labeled dataset into two groups: known classes (plane, ship, and truck) and the unknown class (bird). I then plotted a histogram of the entropy values for both groups and marked the mean of each distribution.

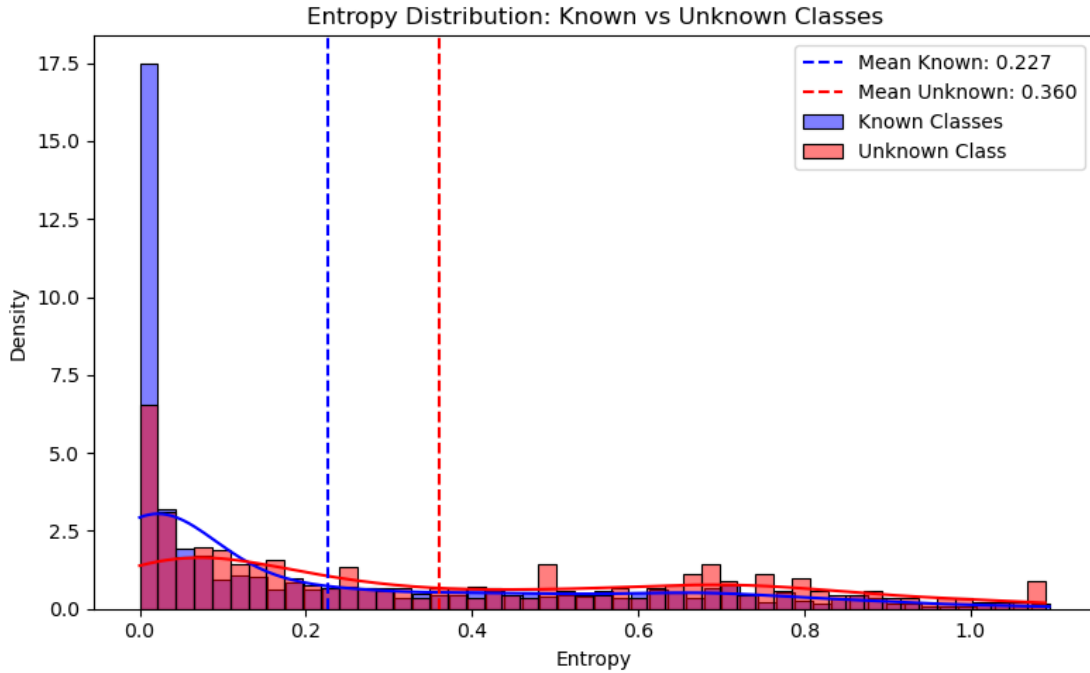


Figure 5: Entropy distributions for known classes and the unknown (bird) class.

Class Group	Mean Entropy	Standard Deviation
Known (Plane/Ship/Truck)	0.2656	0.2895
Unknown (Bird)	0.3448	0.3084

Summary statistics for entropy distributions.

What I observed was consistent with my expectations: the entropy for the unknown class was generally higher. Specifically, the mean entropy for the known classes was approximately 0.266, while for the unknown class it was about 0.345. This suggests that entropy is a useful signal for detecting out-of-distribution samples, although the

distributions do overlap. In the next step, I will use this insight to filter out probable bird images from the unlabeled dataset.

(2e) Filtering Unwanted Images Using Entropy Threshold

Based on the entropy values computed in (2d), I used the ROC curve and Youden’s J statistic to determine a suitable threshold for separating the known classes from the new unwanted class (birds). The best threshold found was:

$$\text{Entropy threshold} = 0.0641$$

To find this threshold, I computed the Receiver Operating Characteristic (ROC) curve using the entropies as scores, and assigned ground truth labels: 0 for known classes and 1 for the bird class. The ROC curve shows the trade-off between true positive rate (TPR) and false positive rate (FPR) at different thresholds. To select the best threshold, I used Youden’s J statistic, defined as:

$$J = \text{TPR} - \text{FPR}$$

This gives the point on the ROC curve that maximizes the difference between correctly detecting birds and mistakenly removing known samples. In other words, it tries to balance sensitivity and specificity, choosing the threshold that provides the most separation between the two groups.

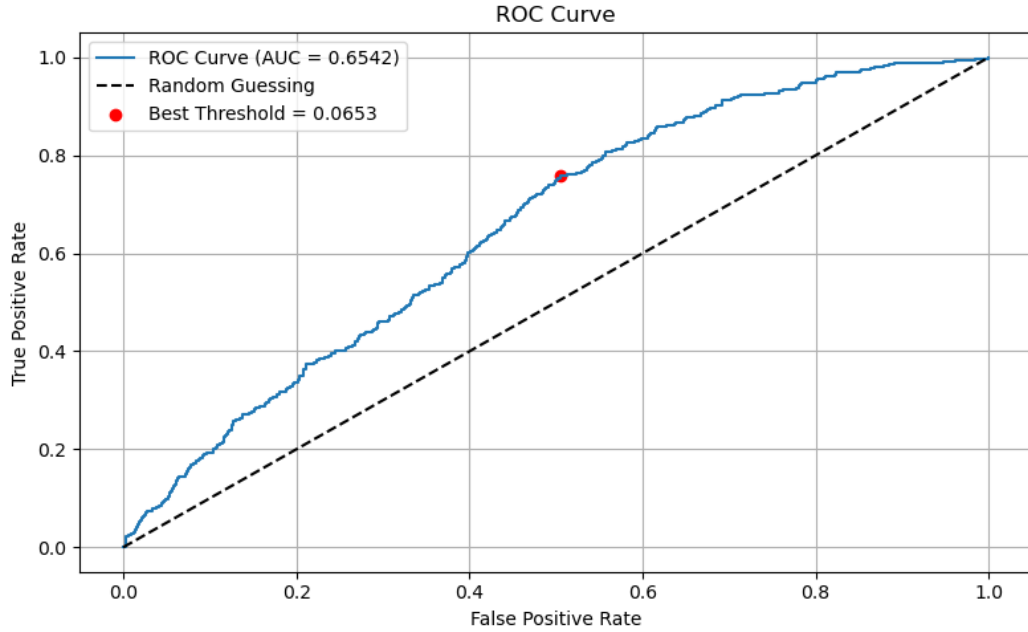


Figure 6: ROC curve using entropy as score. The red dot indicates the threshold chosen by Youden’s J statistic.

I applied the selected threshold to all labeled evaluation samples. If an image’s entropy exceeded the threshold, it was considered “uncertain” and removed. The results are summarized below:

Class	Removed / Total	Percentage
Unwanted class (Birds)	302 / 413	73.12%
Desired classes (Plane/Ship/Truck)	711 / 1187	59.90%

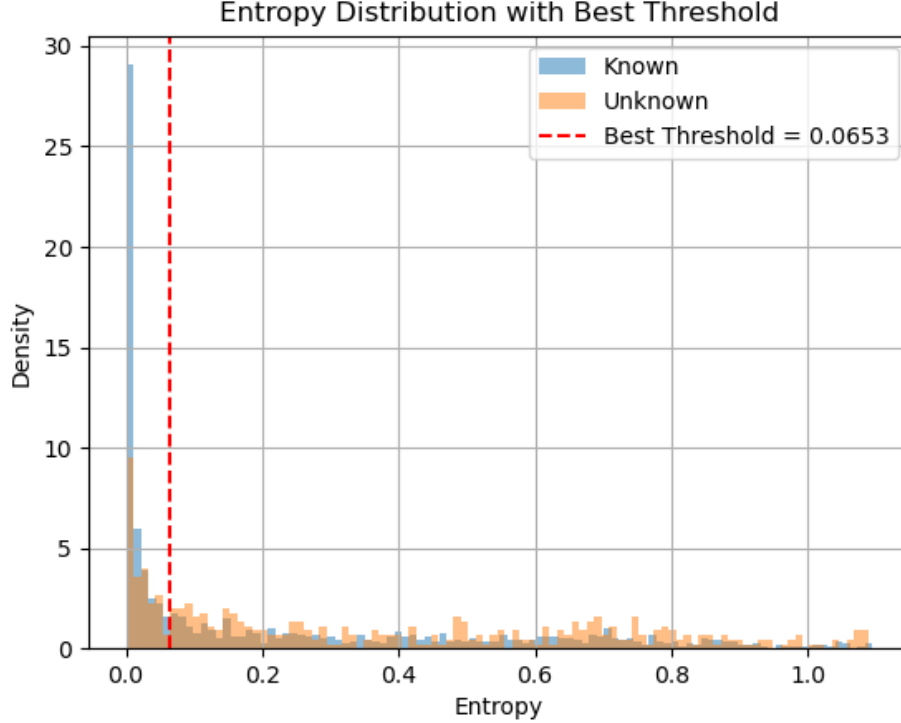


Figure 7: Histogram of entropy values for known and unknown classes. The red vertical line marks the selected threshold.

These figures show that the entropy-based filter successfully removed a large portion of the unwanted class, but at the cost of also removing many images from the known classes. In this case, the threshold trades off recall on birds against precision on known classes. A lower threshold detects more birds but also increases the number of false positives.

Overall, the method shows promise but requires careful tuning to balance the trade-off between removing unwanted classes and retaining known classes. The ROC curve and Youden’s J statistic provided a systematic way to select a threshold, but further experimentation with different thresholds would perhaps give better results.

Alternative Threshold (Midpoint of Means)

As a simpler alternative, I also tested using a threshold equal to the midpoint between the mean entropies of the known and unknown classes:

$$\text{Threshold}_{\text{mean}} = \frac{\text{Mean}_{\text{known}} + \text{Mean}_{\text{unknown}}}{2} = 0.2936$$

Using this threshold, I detected 192 out of 413 bird images (46.49%) and mistakenly removed 365 out of 1187 known samples (30.75%).

Class	Removed / Total	Percentage
Unwanted class (Birds)	192 / 413	46.49%
Desired classes (Plane/Ship/Truck)	365 / 1187	30.75%

Compared to the Youden’s J threshold, this midpoint threshold removes fewer known-class samples, but also detects significantly fewer birds. This method may be useful in settings where retaining known-class data is more critical, and some missed detections of unknowns are acceptable.

This comparison highlights the trade-off between high recall (detecting more birds) and high precision (preserving more known-class samples). The choice of threshold depends on the priorities of the application.

(2f) Final Prediction on Unlabeled Dataset

To finalize the filtering pipeline, I passed the 1600 unlabeled images through the trained VGG11BN model with Monte Carlo Dropout enabled. For each image, I computed the entropy of the mean softmax output from 10 stochastic forward passes and classified it as:

- **1 (unknown class)** if entropy $\geq$ threshold
- **0 (known class)** otherwise

I tested two different thresholds based on the previous analysis:

- **Youden's J threshold** (0.0653): optimized for recall
- **Midpoint between means** (0.2936): more conservative

The resulting binary prediction vectors were saved using `numpy.savez` and submitted as part of the exam. The summary of predictions is shown below:

Threshold Strategy	# Unknowns (1)	# Known (0)
Youden's J (0.0653)	439	1161
Midpoint (0.3052)	256	1344

This comparison shows how a lower threshold captures more of the unknown class at the cost of removing more known samples, while a higher threshold does the opposite. Depending on the application's priorities (e.g., minimizing false negatives vs. preserving data), either could be justified.